

You are my senior C systems engineer, cybersecurity engineer, blockchain engineer, and university assignment reviewer.

I am giving you a university assignment specification for:

****Design and Implementation of a Blockchain-Based Library Book Lending Tracker****

Your job is to help me implement the ENTIRE assignment from A to Z while maximizing compliance with the stated requirements and rubric.

IMPORTANT:

- * Treat the assignment specification and rubric as the authoritative requirements.
- * Do NOT silently omit requirements.
- * Do NOT invent requirements that conflict with the specification.
- * Do NOT take shortcuts that would make the implementation fail during demonstration.
- * Prioritize correctness, security, memory safety, maintainability, explainability, and demonstrability.
- * The final implementation must be something a student can compile, run, test, explain, and defend.
- * Do not merely produce code. First understand and design the system, then implement it systematically.
- * If the specification contains ambiguous terminology or inconsistencies, explicitly identify them and choose a reasonable implementation that remains faithful to the actual requirements.

=====

1. ASSIGNMENT REQUIREMENTS

=====

The system must implement:

1. A blockchain data structure containing lending records.
2. A book registry loaded from books.txt.
3. A member registry loaded from members.txt.
4. Validation of book IDs and member IDs before lending actions.
5. SHA-256 hashing to link blocks and protect integrity.
6. Digital signatures using ECDSA to authenticate lending actions.
7. Chain validation to detect tampering.
8. File-based persistence.
9. CLI functionality for borrowing, returning, and viewing lending records.
10. Proper error handling and data validation.
11. Tamper detection demonstration.

The required registry formats are:

books.txt:

BK001,Things Fall Apart,Chinua Achebe
BK002,Americanah,Chimamanda Ngozi Adichie
BK003,The River Between,Ngũgĩ wa Thiong'o

Format:
book_id,title,author

members.txt:

ALU001,John Doe,BLK101
ALU002,Jane Smith,BLK101
ALU003,Amara Diallo,BLK101

Format:
member_id,full_name,course_code

The program must:

- * Load all books into an array of Book structs at startup.
- * Load all members into an array of Member structs at startup.
- * Reject invalid book IDs.
- * Reject invalid member IDs.
- * Detect missing books.txt.
- * Detect missing members.txt.
- * Detect empty registry files.

Required structures include:

Book:
char book_id[20];
char title[80];
char author[50];

Member:
char member_id[20];
char full_name[50];
char course_code[10];

The block must contain:

- * index
- * timestamp
- * book_id
- * book_title
- * member_id
- * member_name
- * action

- * previous_hash
- * signature
- * hash

Action values:

- * BORROWED
- * RETURNED
- * OVERDUE

previous_hash must contain the SHA-256 hash of the previous block.

hash must be the SHA-256 hash of the appropriate block data.

The signature must be an ECDSA digital signature of the block data.

The chain must begin with a genesis block:

- * index = 0
- * previous_hash = 64 zeros

=====

2. REQUIRED FUNCTIONAL BEHAVIOR

=====

IMPLEMENT:

A. Registry Loading

At startup:

books.txt -> Book array
members.txt -> Member array

Validate:

- * file exists
- * file opens successfully
- * file is not empty
- * records are valid
- * fields fit safely inside buffers
- * malformed records are handled without crashing

B. Borrow

When the user requests:

BORROW <book_id> <member_id>

The program must:

1. Validate the book ID.
2. Validate the member ID.
3. Determine whether the book is currently on loan.
4. Reject borrowing if already borrowed.
5. Obtain the canonical book information from the registry.
6. Obtain the canonical member information from the registry.
7. Create a new block.
8. Set the action to BORROWED.
9. Set the timestamp.
10. Set the previous_hash.
11. Calculate the block hash.
12. Create an ECDSA signature.
13. Append the block to the blockchain.
14. Persist the updated blockchain.
15. Report success clearly.

C. Return

When the user requests:

RETURN <book_id>

The program must:

1. Validate the book ID.
2. Search the blockchain for the most recent relevant record.
3. Determine whether the book is currently borrowed.
4. Reject returning a book that is not currently borrowed.
5. Create a RETURNED block.
6. Preserve the relevant book/member information.
7. Set previous_hash correctly.
8. Hash the block.
9. Sign the block.
10. Append it.
11. Persist it.
12. Report success.

D. View Records

Display useful information including:

- * index
- * book title
- * book ID
- * member name

- * member ID
- * action
- * timestamp
- * hash
- * previous hash
- * signature validity

Make the output readable in a terminal.

E. Validate Chain

The validation routine must verify:

1. Genesis block correctness.
2. Every block's stored hash against a freshly calculated hash.
3. Every block's previous_hash against the previous block's hash.
4. Signature validity where appropriate.
5. Block ordering/index consistency if applicable.

It must clearly report whether the chain is valid or compromised.

F. Tamper Detection

The assignment explicitly requires demonstrating that modifying a past block causes validation failure.

Design the persistence format so that:

1. The chain can be saved to disk.
2. The saved data can be modified externally.
3. A single-byte modification can be demonstrated.
4. Reloading the chain causes validation to fail.
5. The program clearly identifies that tampering/integrity failure occurred.

=====

3. CRYPTOGRAPHY REQUIREMENTS

=====

Use SHA-256 correctly.

Use ECDSA correctly.

Design the cryptographic flow carefully.

Explain:

- * what exactly is hashed
- * what exactly is signed

- * what the private key does
- * what the public key does
- * how signatures are verified
- * why changing block data invalidates the hash
- * why changing an earlier block breaks subsequent previous_hash relationships
- * how persistence interacts with cryptographic verification

Do NOT store private keys inside source code.

Do NOT print private keys unnecessarily.

Do NOT confuse hashing with encryption.

Do NOT confuse digital signatures with hashes.

Prefer a well-established cryptographic library such as OpenSSL if compatible with the assignment environment.

If using OpenSSL:

- * identify required headers
- * identify required libraries
- * provide exact compilation/linking instructions
- * use appropriate modern APIs where practical
- * avoid deprecated APIs where a reasonable alternative exists

=====

4. MEMORY SAFETY AND C QUALITY

=====

This is a C assignment.

Write high-quality defensive C.

Pay particular attention to:

- * buffer sizes
- * null termination
- * malloc/calloc/realloc failures
- * free()
- * file handles
- * parsing errors
- * integer conversion
- * string handling
- * malformed input
- * memory leaks
- * double frees
- * use-after-free

- * buffer overflows
- * format-string vulnerabilities
- * unsafe input functions
- * signed/unsigned issues
- * error propagation

Avoid unsafe patterns such as blindly using strcpy(), strcat(), gets(), or unchecked scanf().

Use clear modular functions.

Use meaningful names.

Keep functions reasonably focused.

Use headers properly.

Use const where appropriate.

Document security-sensitive code.

=====

5. BLOCKCHAIN DESIGN

=====

Design the blockchain as a linked list as required by the assignment concept.

Clearly define:

Block

Blockchain

Explain:

Genesis block

Current block

Previous block

Hash

Previous hash

Digital signature

Chain traversal

Appending a block

Validation

Use dynamic memory responsibly.

Make sure the linked-list representation and persistent representation cannot accidentally create inconsistent state.

=====

6. PERSISTENCE

=====

The blockchain must survive program termination.

Design a robust persistence strategy.

The persisted chain must be reloadable.

On startup:

1. Load registries.
2. Load blockchain if it exists.
3. If no blockchain exists, create the genesis block.
4. Validate the loaded chain.
5. Do not blindly trust persisted data.
6. Reject or clearly flag corrupted/tampered data.

The persistence design must support the assignment's tamper-detection demonstration.

Explain why the chosen serialization format is appropriate.

If binary persistence is used, document the exact structure and implications of binary serialization.

Be careful about:

- * struct padding
- * portability
- * endianness
- * fixed-width types
- * serialized sizes

If raw structs are written directly to disk, explicitly assess whether this is safe and appropriate.

=====

7. CLI DESIGN

=====

Create a clean CLI.

At minimum support:

- * loading/initialization
- * borrow
- * return

- * view records
- * validate chain
- * exit

If useful, also support:

- * help
- * list books
- * list members
- * chain status

Make invalid commands produce helpful messages.

Do not allow invalid IDs to silently pass.

=====

8. PROJECT STRUCTURE

=====

Before writing code, propose a professional project structure.

For example:

```
src/  
main.c  
blockchain.c  
blockchain.h  
registry.c  
registry.h  
crypto.c  
crypto.h  
persistence.c  
persistence.h  
cli.c  
cli.h  
validation.c  
validation.h
```

```
include/  
...
```

```
data/  
books.txt  
members.txt
```

```
keys/  
...
```

tests/

...

Makefile

README.md

.gitignore

You may modify this structure if you have a better justified design.

Explain why every major file exists.

=====

9. TESTING

=====

Create a systematic test plan.

At minimum test:

REGISTRY:

- * valid books.txt
- * valid members.txt
- * missing books.txt
- * missing members.txt
- * empty books.txt
- * empty members.txt
- * malformed lines
- * duplicate IDs if relevant
- * oversized fields

BORROW:

- * valid book + valid member
- * invalid book
- * invalid member
- * both invalid
- * borrowing already borrowed book

RETURN:

- * valid return
- * returning never-borrowed book
- * returning already-returned book
- * invalid book ID

BLOCKCHAIN:

- * genesis block
- * multiple blocks
- * correct previous_hash
- * correct hash
- * index sequence
- * persistence and reload

CRYPTOGRAPHY:

- * valid signature
- * invalid signature
- * modified signed data
- * wrong public key

TAMPERING:

- * modify block data
- * modify stored hash
- * modify previous_hash
- * modify one byte in persistence file
- * reload
- * validate
- * demonstrate failure

MEMORY:

- * run with sanitizers where possible
- * check for leaks
- * check invalid memory access

=====

10. RUBRIC OPTIMIZATION

=====

The rubric has three major scored areas plus submission compliance.

DO NOT simply claim that the rubric is satisfied.

Create a REQUIREMENT TRACEABILITY MATRIX.

For every requirement, show:

Requirement
 Implementation location
 How it is tested
 How it is demonstrated
 Evidence to capture for report/video

Specifically verify:

A. Compliance with Submission Requirements

Ensure the final submission includes:

- * complete C source code
- * GitHub repository
- * demo video
- * technical report
- * README
- * compilation instructions
- * build/run instructions
- * dependencies
- * screenshots
- * system design diagram
- * challenges and solutions

B. Technical Correctness and Code Quality

Verify:

- * correct structures
- * correct blockchain logic
- * correct registry logic
- * correct SHA-256
- * correct persistence
- * correct validation
- * correct error handling
- * memory safety
- * readable modular C

C. Security and Cryptographic Keys

Verify:

- * ECDSA implementation
- * private/public key handling
- * signature generation
- * signature verification
- * integrity validation
- * tamper detection
- * key protection
- * no hard-coded private key

D. Demo Video

The demo must clearly show:

1. Program startup.
2. Registry loading.
3. Valid IDs.
4. Invalid IDs.
5. Borrow operation.
6. Return operation.
7. Viewing records.
8. Chain validation.
9. Persistence/reload.
10. Tamper demonstration.
11. Chain validation failing after tampering.

Create a 3–5 minute demo script with exact actions and narration.

=====

11. TECHNICAL REPORT

=====

Produce a report outline and later the final report containing:

1. Introduction
2. Problem statement
3. System objectives
4. System architecture
5. Blockchain implementation
6. Block structure
7. Registry system
8. SHA-256 integrity mechanism
9. ECDSA authentication
10. Key management
11. Persistence mechanism
12. Chain validation
13. Tamper detection
14. Error handling
15. Testing
16. Screenshots
17. Challenges encountered
18. Solutions
19. Limitations
20. Conclusion

Include a system design diagram showing:

books.txt
|
v
Book Registry ----

members.txt -----> Validation ---> Lending Action

|

v

New Block

|

SHA-256 + ECDSA

|

v

Blockchain

|

v

chain file

Make the final architecture technically accurate to the implementation.

=====

12. README

=====

Create a professional README containing:

- * project title
- * description
- * features
- * architecture
- * requirements
- * dependencies
- * installation
- * compilation
- * execution
- * CLI commands
- * test procedure
- * tamper-detection procedure
- * cryptographic design
- * project structure
- * troubleshooting

A new student should be able to clone the repository and run the project by following the README.

=====

13. GITHUB/SUBMISSION QUALITY

=====

Prepare:

.gitignore

Do not commit:

- * private keys
- * generated binaries
- * temporary files
- * secrets
- * unnecessary build artifacts

Include example/public test data where appropriate.

Use sensible Git commits.

Before finalizing, check that the repository contains everything required for another person to build and test the project.

=====
14. IMPLEMENTATION WORKFLOW
=====

DO NOT jump immediately into one huge code dump.

Work in phases:

PHASE 1 — Analyze assignment

- * extract every requirement
- * identify ambiguities
- * identify risks
- * create requirement checklist

PHASE 2 — Architecture

- * project structure
- * data structures
- * module responsibilities
- * data flow
- * cryptographic flow
- * persistence strategy

PHASE 3 — Core C foundation

- * structs
- * registry loading
- * validation
- * memory management

PHASE 4 — Blockchain

- * genesis
- * append
- * linked list
- * hash calculation
- * chain validation

PHASE 5 — Cryptography

- * SHA-256
- * ECDSA keys
- * signing
- * verification

PHASE 6 — Persistence

- * save
- * load
- * corruption detection
- * tamper demonstration

PHASE 7 — CLI

- * commands
- * input validation
- * user feedback

PHASE 8 — Testing

- * positive tests
- * negative tests
- * security tests
- * tamper tests
- * memory-safety tests

PHASE 9 — Documentation

- * README
- * report
- * architecture diagram
- * test evidence

PHASE 10 — Rubric audit

- * inspect every rubric item
- * identify anything missing
- * fix it
- * produce final traceability matrix

=====

15. IMPORTANT STUDENT-UNDERSTANDING MODE

=====

I am a student, so do not hide the engineering logic from me.

For every major component explain:

1. What it does.
2. Why it is needed.
3. What data enters it.
4. What data comes out.
5. What could go wrong.
6. How it connects to other components.
7. Which assignment requirement it satisfies.
8. Which rubric item it supports.

Do not assume I understand blockchain, SHA-256, ECDSA, linked lists, serialization, or cryptographic signatures.

Teach these concepts as we build them.

=====

16. FINAL QUALITY GATE

=====

Before declaring the project complete, perform a strict final audit.

Pretend you are the lecturer grading the submission.

Check every requirement independently.

For each requirement answer:

PASS / NEEDS FIX

Then explain exactly why.

Do NOT give the project a fabricated numerical score.

Instead produce:

1. Requirements checklist
2. Rubric traceability matrix
3. Security audit
4. Memory-safety audit
5. Testing matrix

6. Demo checklist
7. Report checklist
8. README checklist
9. Remaining issues
10. Exact fixes required

Only after every requirement has been addressed should you call the implementation complete.

Most importantly:

****DO NOT optimize for merely producing code. Optimize for producing a correct, secure, demonstrable, explainable, rubric-compliant university project.****